المدرسة الوطنية العليا للذكاء الاصطناعي وعلوم المعطيات - تارودانت

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵜⴰⵏⴰⵏⵉⵏⵜ ⵏ ⵓⵙⵙⵏⵉⵙ ⴰⴷⵅⵓⵏ ⴷ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ ⵏ ⵜⵓⵙⵙⴰⵏⵜ - ⵜⴰⵔⵓⴷⴰⵏⵜ

ÉCOLE NATIONALE SUPÉRIEURE DE L'INTELLIGENCE ARTIFICIELLE ET SCIENCES DES DONNÉES – TAROUDANT

Cahier de Recette Comparatif

Agadir Tsunami-Ready

Systeme d'Alerte et d'Evacuation Cotiere

Version IA (Augmenteds) **vs** Version Manuelle (Architects)

Module	SIBD — Systemes d'Information et Bases de Donnees
Enseignant	Pr. S. El-Ateif
Equipe	Equipe 8 — Tsunami-Ready
Role	QA Engineer
Technologies IA	Node.js · Flask · JavaScript Vanilla · MySQL
Technologies Manuel	Django · Python · SQLite · Google Maps API
Date	Avril 2026
Nombre de tests	27 scenarios au total (15 IA + 12 Manuel)

Version Augmenteds — IA	
11 PASSES	**1 BUG CRITIQUE · 3 PARTIELS**
Taux de reussite : **73%** (11/15)	

Version Architects — Manuel	
7 PASS	**4 FAIL · 1 EN ATTENTE**
Taux de reussite : **58%** (7/12)	

Table des matières

1 Introduction Generale **2**
1.1 Objet du document . . . 2
1.2 Perimetre . . . 2
1.3 Environnements de test . . . 2
1.4 Conventions de statut . . . 2

2 Tableaux Recapitulatifs Comparatifs **3**
2.1 Version Augmenteds — IA (15 tests) . . . 3
2.2 Version Architects — Manuel (12 tests) . . . 3

3 Partie I — Tests Version Augmenteds (IA) **5**
Categorie 1 — Sirenes et Triggers . . . 5
Categorie 2 — Authentification . . . 17
Categorie 3 — Securite RBAC . . . 21
Categorie 4 — Anti-injection SQL . . . 24
Categorie 5 — GPS et Evacuation . . . 25

4 Partie II — Tests Version Architects (Manuel) **29**
[Architects] Categorie 1 — Sirenes et Triggers . . . 29
[Architects] Categorie 2 — Authentification . . . 33
[Architects] Categorie 3 — Securite RBAC . . . 36
[Architects] Categorie 4 — Anti-injection SQL . . . 38
[Architects] Categorie 5 — GPS et Evacuation . . . 39

5 Bilan des Anomalies **41**
5.1 Anomalies — Version Augmenteds (IA) . . . 41
5.2 Anomalies — Version Architects (Manuel) . . . 41

6 Partie III — Analyse Comparative **42**
6.1 Tableau de synthese comparatif . . . 42
6.2 Analyse par domaine . . . 43
6.3 Bilan comparatif global . . . 45

7 Tableau de Synthese Final **46**

8 Conclusion et Recommandations **48**
8.1 Points forts identifies . . . 48
8.2 Corrections prioritaires avant la soutenance . . . 48
8.3 Conclusion comparative . . . 48

Introduction Generale

Objet du document

Ce cahier de recette **comparatif** documente et analyse les tests de validation realises sur les deux versions du projet **Agadir Tsunami-Ready** : la version **Augmenteds** (generee par intelligence artificielle) et la version **Architects** (developpee manuellement). Il constitue le livrable officiel du pole QA Engineer, conformement au cahier des charges du projet SIBD 2025-2026.

Perimetre

Les tests couvrent les fonctionnalites communes aux deux versions : declenchement des sirenes d'alerte, authentification des utilisateurs, controle d'acces (RBAC), protection contre les injections SQL, et calcul des routes d'evacuation GPS. La version Augmenteds a fait l'objet de tests dynamiques (execution reelle), tandis que la version Architects a ete evaluee par analyse statique du code source, en l'absence de captures d'ecran.

Environnements de test

Element	Augmenteds (IA)	Architects (Manuel)
URL application	`http://localhost:8080`	`http://localhost:8000`
Backend	Node.js `:3001` + Flask `:5000`	Django (Python)
Base de donnees	MySQL 8	SQLite
Frontend	JavaScript Vanilla	Django Templates
Navigateur	Microsoft Edge (Chromium)	Analyse statique
Authentification	JWT (localStorage)	Django auth sessions
Methode de test	Tests dynamiques + screenshots	Analyse statique du code

Conventions de statut

PASSE	(IA) Test dynamique conforme aux attendus
BUG CONFIRME	(IA) Bug critique confirme en execution
PARTIEL	(IA) Test partiellement valide, anomalie non bloquante
FAIL	(Manuel) Echec identifie par analyse statique
EN ATTENTE	(Manuel) A valider manuellement, non verifiable statiquement

Tableaux Recapitulatifs Comparatifs

Version Augmenteds — IA (15 tests)

Categorie	ID	Description	Priorite	Statut
Sirenes	TEST-01	Sirenes declenchees si magnitude ≥ 6.5 marine	Critique	PASSE
	TEST-02	Sirenes non declenchees si magnitude < 6.5	Critique	PASSE
	TEST-03	Seuil exact 6.5 + temps reponse < 1 s	Critique	PASSE
	TEST-04	Bug champ `type` absent dans les donnees	Critique	BUG CONFIRME
	TEST-05	Reinitialisation des sirenes apres alerte	Haute	PARTIEL
	TEST-06	KPIs du dashboard mis a jour en temps reel	Haute	PASSE
Auth.	TEST-07	Connexion avec compte valide	Critique	PASSE
	TEST-08	Connexion avec mauvais mot de passe	Critique	PASSE
	TEST-09	Acces sans token JWT — redirection	Critique	PASSE
	TEST-10	Inscription d'un nouveau citoyen	Haute	PASSE
RBAC	TEST-11	Citoyen ne peut PAS activer une sirene	Critique	PARTIEL
	TEST-12	Admin PEUT activer une sirene via l'API	Critique	PASSE
SQL	TEST-13	Injection SQL bloquee dans le formulaire	Critique	PASSE
GPS	TEST-14	GPS en zone Agadir — route calculee	Haute	PASSE
	TEST-15	GPS hors zone — comportement	Haute	PARTIEL
BILAN			**11 PASSES · 1 BUG · 3 PARTIELS**	

Version Architects — Manuel (12 tests)

ID	Description	Remarque principale	Statut
TEST-01	Sirenes declenchees mag 7.2 marine	Logique correcte : magnitude > 6.5 ET en_mer=True	PASSE
TEST-02	Sirenes non declenchees mag 6.4	6.4 < 6.5 : aucune sirene. Correct.	PASSE
TEST-03	Temps de reponse < 1 seconde	Traitement synchrone. A mesurer manuellement.	EN ATTENTE
TEST-04	Bug champ `en_mer` manquant	BUG : events_import.json sans en_mer	FAIL
TEST-05	Login valide admin	Django auth standard. Redirection confirmee.	PASSE
TEST-06	Login mauvais mot de passe	Erreur de formulaire affichee. Correct.	PASSE
TEST-07	Acces sans token — redirection	@login_required + LOGIN_URL protege.	PASSE
TEST-08	Citoyen bloque activation sirene	Aucun controle de role. Endpoint inexistant.	FAIL
TEST-09	Admin peut activer sirene	Endpoint /api/sirens/activate/ inexistant.	FAIL
TEST-10	Injection SQL bloquee	ORM Django, requetes parametrees.	PASSE
TEST-11	GPS Agadir — route calculee	Logique correcte. Necessite cle Google Maps.	PASSE
TEST-12	GPS hors zone — message erreur	Pas de verification zone geo. Echec.	FAIL
BILAN			**7P · 4F · 1A**

Partie I — Tests Version Augmenteds (IA)

Version Augmenteds — Generee par Intelligence Artificielle

Node.js · Flask · JavaScript Vanilla · MySQL · Tests dynamiques avec execution reelle

Categorie 1 — Sirenes et Triggers

Sirenes & Triggers

Tests du declenchement automatique, seuils de magnitude, reinitialisation et KPIs temps reel.

TEST-01 — Les sirenes se declenchent si seisme dangereux

ID	Categorie	Priorite	Prof	Statut
TEST-01	Sirenes & Triggers	Critique	Oui — impose par le prof	PASSE

Objectif et utilite du test

Verifier que le systeme declenche automatiquement toutes les sirenes lorsqu'un seisme marin dangereux est detecte. La condition dans `app.js` doit activer les sirenes pour toute magnitude ≥ 6.5 avec type `marine`.

Etapes d'execution

1. Ouvrir le dashboard sur `http://localhost:8080`, section **Sirenes & Alertes**
2. Verifier que les 4 sirenes sont en "En veille"
3. F12 → Sources → Snippets → executer :

```
handleSeismicEvent({ magnitude: 7.2, type: 'marine',
  latitude: 30.31, longitude: -9.88, location: 'Atlantique'
    });
```

4. Observer la page "Sirenes & Alertes" et le bandeau en haut

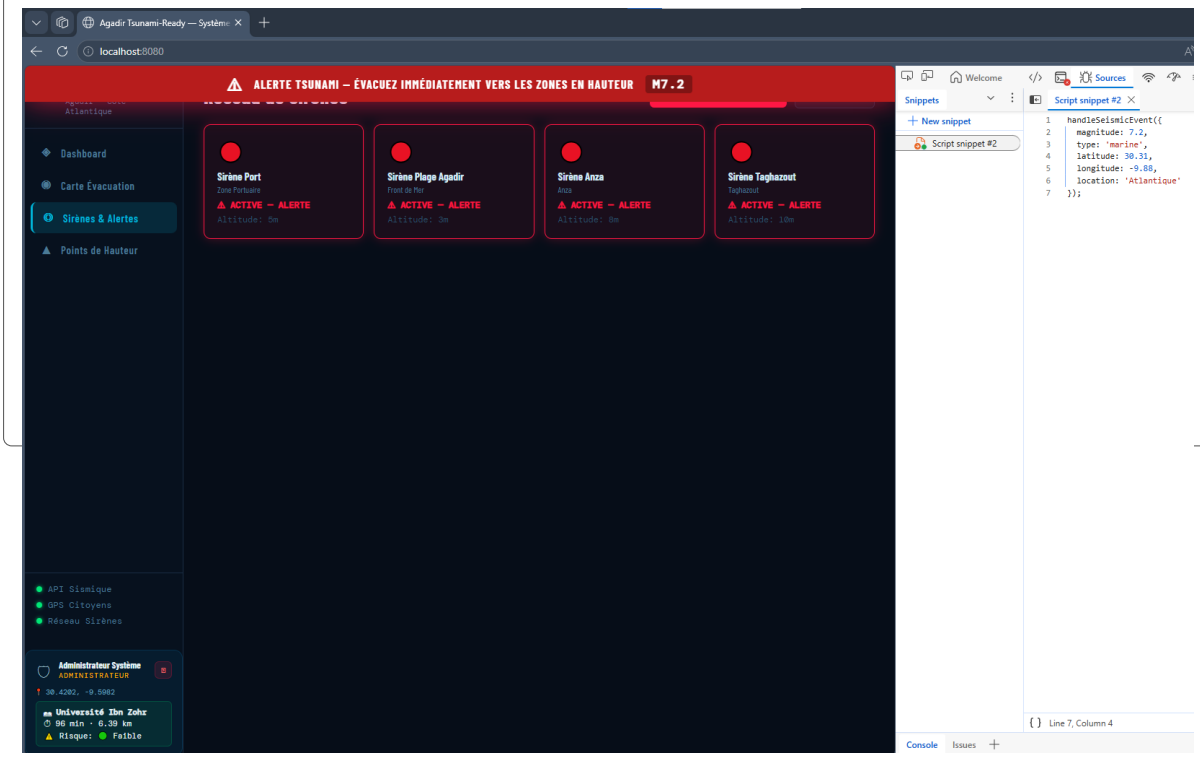
Succes : 4 sirenes en rouge "ACTIVE — ALERTE". Bandeau "ALERTE TSUNAMI — M7.2". KPI "Sirenes actives" = 4.

Resultat obtenu

Statut : PASSE

4 sirenes en "ACTIVE — ALERTE". Bandeau rouge M7.2 apparu. Console : *Broadcast WhatsApp : 0 envoyes, 5 echoues* (Twilio non configure, hors perimetre).

Captures d'ecran — TEST-01 : Sirenes actives apres magnitude 7.2 ·



TEST-02 — Les sirenes ne sonnent PAS pour un seisme faible

ID	Categorie	Priorite	Prof	Statut
TEST-02	Sirenes & Triggers	Critique	Oui — impose par le prof	PASSE

Objectif et utilite du test

Verifier qu'aucune fausse alerte n'est declenchee pour une magnitude inferieure a 6.5.

Etapes d'execution

1. Cliquer "Reinitialiser"
2. Modifier le snippet : `magnitude: 6.4`
3. Executer et observer les 4 cartes sirenes

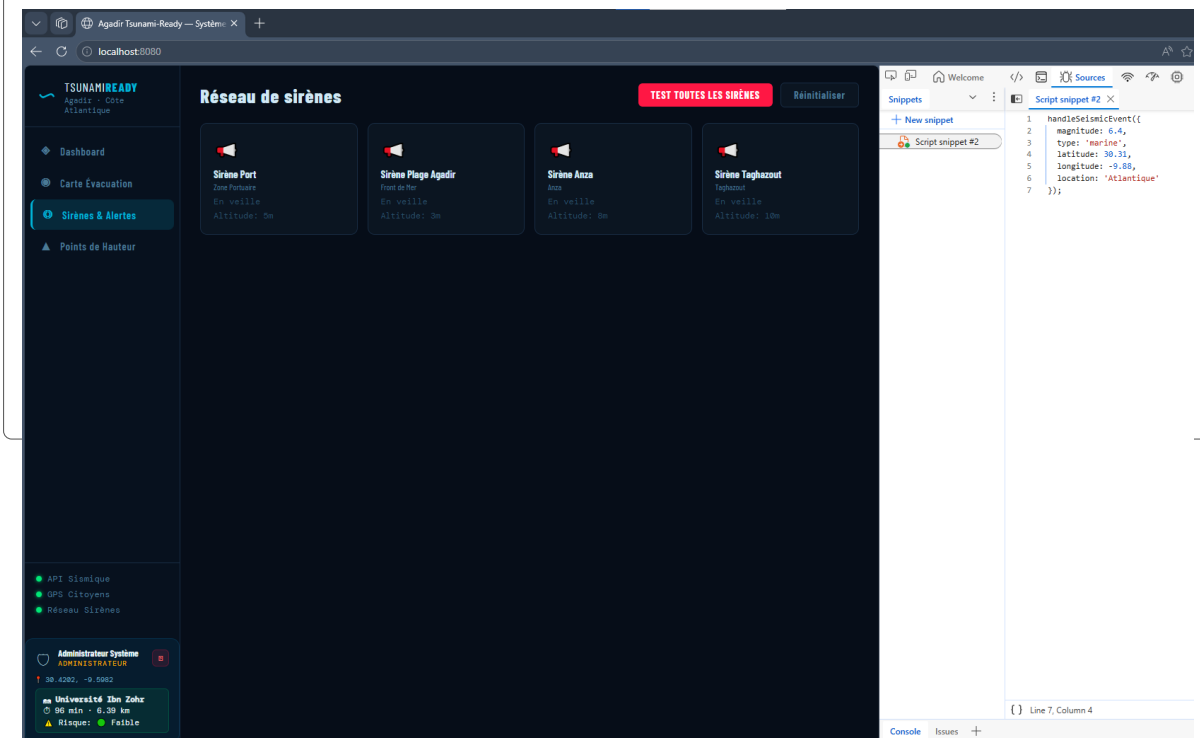
Succes : 4 sirenes restent "En veille". Aucun bandeau rouge. KPI "Sirenes actives" = 0.

Resultat obtenu

Statut : PASSE

4 sirenes en veille apres seisme de 6.4. Aucun bandeau. Un cercle cartographique normal apparu.

Captures d'ecran — TEST-02 : Sirenes en veille avant/apres magnitude 6.4 ·



TEST-03 — Seuil exact 6.5 et temps de reponse inferieur a 1 seconde

ID	Categorie	Priorite	Prof	Statut
TEST-03	Sirenes & Triggers	Critique	Oui — exigence explicite du prof	PASSE

Objectif et utilite du test

(1) Le seuil 6.5 doit etre inclusif. (2) Le temps de reponse doit etre < 1 seconde.

Etapes d'execution

1. Reinitialiser. Ouvrir F12 → Network
2. Executer avec `magnitude: 6.5, type: 'marine'`
3. Mesurer le temps de reponse. Repeter 5 fois.

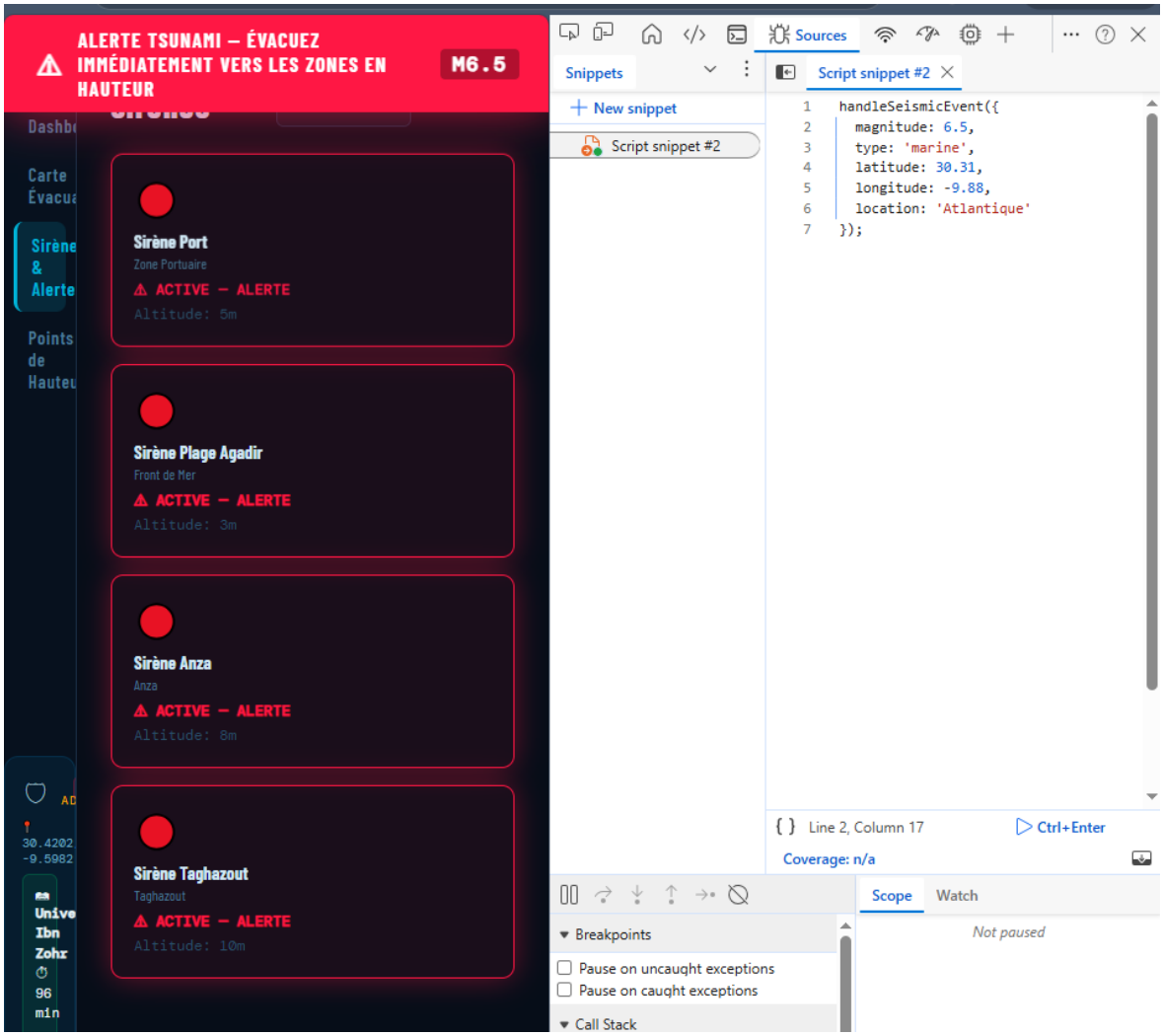
Resultat obtenu

Statut : PASSE

Essai	1	2	3	4	5	Moyenne
Temps	<1s	<1s	<1s	<1s	<1s	**< 1 seconde**

Network : `whatsapp-broadcast` 3–19 ms · `events?limit=20` 16–337 ms · DOMContentLoaded 251 ms.

Captures d'ecran — TEST-03 : Onglet Network + sirenes a magnitude 6.5 ·



Network

Preserve log | Disable cache | No throttling

Filter | Invert | More filters

All | Fetch/XHR | Doc | CSS | JS | Font | Img | Media | Manifest | Socket | Wasm | Other

20 ms | 40 ms | 60 ms | 80 ms | 100 ms

Name	Status	Type	Initiator	Size	Time	Fulfill...
HTx3L3I-JCGChYJ8VI-L6...	200	font	css2?family=B	0 B	0 ms	(mem...
HTxwL3I-JCGChYJ8VI-L...	200	font	css2?family=B	0 B	0 ms	(mem...
HTxwL3I-JCGChYJ8VI-L...	200	font	css2?family=B	0 B	0 ms	(mem...
HTxwL3I-JCGChYJ8VI-L...	200	font	css2?family=B	0 B	0 ms	(mem...
aFTU7PB1QTsUX8KYthq...	200	font	css2?family=B	0 B	0 ms	(mem...
aFTR7PB1QTsUX8KYvu...	200	font	css2?family=B	0 B	0 ms	(mem...
events?limit=20	200	fetch	api.js:18	2.6 kB	36 ms	
status	204	prefli...	Preflight	0 B	4 ms	
active	204	prefli...	Preflight	0 B	4 ms	
status	304	fetch	api.js:18	211 B	8 ms	
active	304	fetch	api.js:18	210 B	11 ms	
ws	101	webs...	websocket.js:1	0 B	Pendi...	
whatsapp-broadcast	204	prefli...	Preflight	0 B	4 ms	
whatsapp-broadcast	200	fetch	sirens.js:20	678 B	12 ms	
events?limit=20	200	fetch	api.js:18	2.6 kB	318 ms	
events?limit=20	200	fetch	api.js:18	2.6 kB	10 ms	
events?limit=20	200	fetch	api.js:18	2.6 kB	320 ms	
events?limit=20	200	fetch	api.js:18	2.6 kB	9 ms	
events?limit=20	200	fetch	api.js:18	2.6 kB	343 ms	
events?limit=20	200	fetch	api.js:18	2.6 kB	8 ms	
whatsapp-broadcast	204	prefli...	Preflight	0 B	3 ms	
whatsapp-broadcast	200	fetch	sirens.js:20	678 B	9 ms	
whatsapp-broadcast	204	prefli...	Preflight	0 B	3 ms	
whatsapp-broadcast	200	fetch	sirens.js:20	678 B	18 ms	
whatsapp-broadcast	200	fetch	sirens.js:20	678 B	10 ms	
whatsapp-broadcast	200	fetch	sirens.js:20	678 B	13 ms	
whatsapp-broadcast	204	prefli...	Preflight	0 B	3 ms	
whatsapp-broadcast	200	fetch	sirens.js:20	678 B	9 ms	
events?limit=20	200	fetch	api.js:18	2.6 kB	319 ms	
events?limit=20	200	fetch	api.js:18	2.6 kB	10 ms	

49 requests 29.6 kB transferred 603 kB resources Finish: 4 min

TEST-04 — Bug : champ type manquant dans les donnees reelles

ID	Categorie	Priorite	Prof	Statut
TEST-04	Sirenes & Triggers	Critique	Non — test supplementaire	BUG CONFIRME

Objectif et utilite du test

Identifier un bug fondamental : `events_import.json` ne contient aucun champ `type`, donc la condition `type === 'marine'` ne sera jamais vraie en production.

Etapes d'execution

1. Reinitialiser. Creer un snippet sans champ `type` :

```
handleSeismicEvent({ magnitude: 7.2,
  latitude: 30.31, longitude: -9.88, location: 'Atlantique'
  // Pas de champ 'type' -- simule events_import.json
});
```

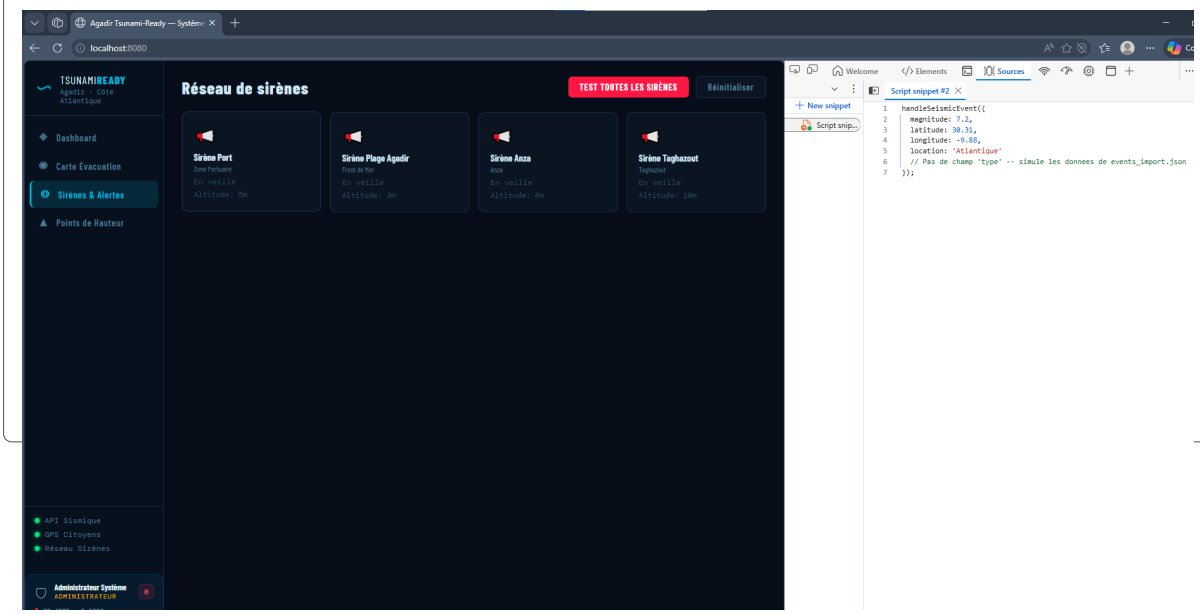
2. Executer et observer les sirenes

Anomalie / Bug identifie

Bug confirme : Sirenes restees "En veille" malgre magnitude 7.2.

Localisation : `app.js` — fonction `handleSeismicEvent()`
Code fautif : `if (event.magnitude >= 6.5 && event.type === 'marine')`
Correction : `if (event.magnitude >= 6.5 && (!event.type || event.type === 'marine`
Impact : `events_import.json` ne declenche jamais les sirenes auto.

Captures d'ecran — TEST-04 : Sirenes en veille malgre magnitude 7.2 sans champ type ·



TEST-05 — Reinitialisation du reseau de sirenes

ID	Categorie	Priorite	Prof	Statut
TEST-05	Sirenes & Triggers	Haute	Non — nouveau test	PARTIEL

Objectif et utilite du test

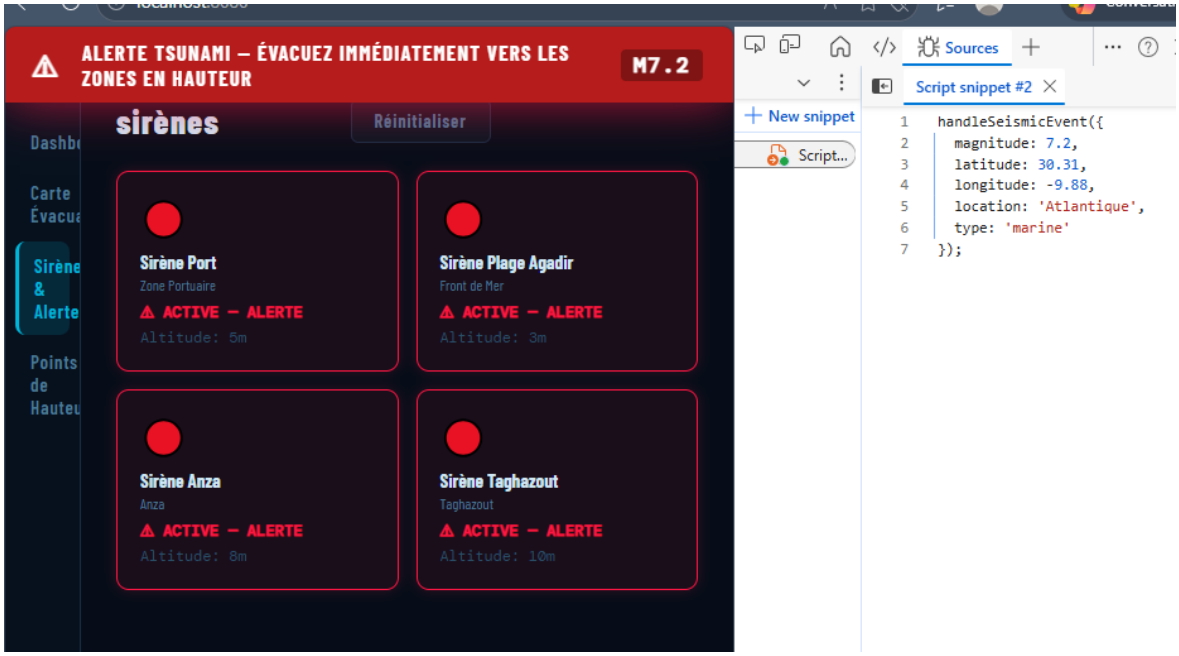
Verifier que le bouton “Reinitialiser” remet toutes les sirenes en veille et que l’etat persiste apres rechargement.

Anomalie / Bug identifie

Anomalie detectee :

Observe : 3/4 sirenes remises en veille. Apres Ctrl+R : Sirene Port reste “ACTIVE”.
Cause : `resetAll()` met a jour uniquement le Store local, sans appel API.
Correction : Ajouter `fetch('/api/sirens/S1/deactivate')` dans `resetAll()`.

Captures d’ecran — TEST-05 : Apres Reinitialiser + apres Ctrl+R ·



Réseau de sirènes
TEST TOUTES LES SIRÈNES
Réinitialiser
Dashbo
Carte Évacua
Sirène & Alerte
Points de Hauteu
Sirène Port
Zone Portuaire
ACTIVE — ALERTE
Altitude: 5m
Sirène Plage Agadir
Front de Mer
En veille
Altitude: 3m
Sirène Anza
Anza
En veille
Altitude: 8m
Sirène Taghazout
Taghazout
En veille
Altitude: 10m

TEST-06 — KPIs du dashboard mis a jour en temps reel

ID	Categorie	Priorite	Prof	Statut
TEST-06	Sirenes & Triggers	Haute	Non — nouveau test	PASSE

Objectif et utilite du test

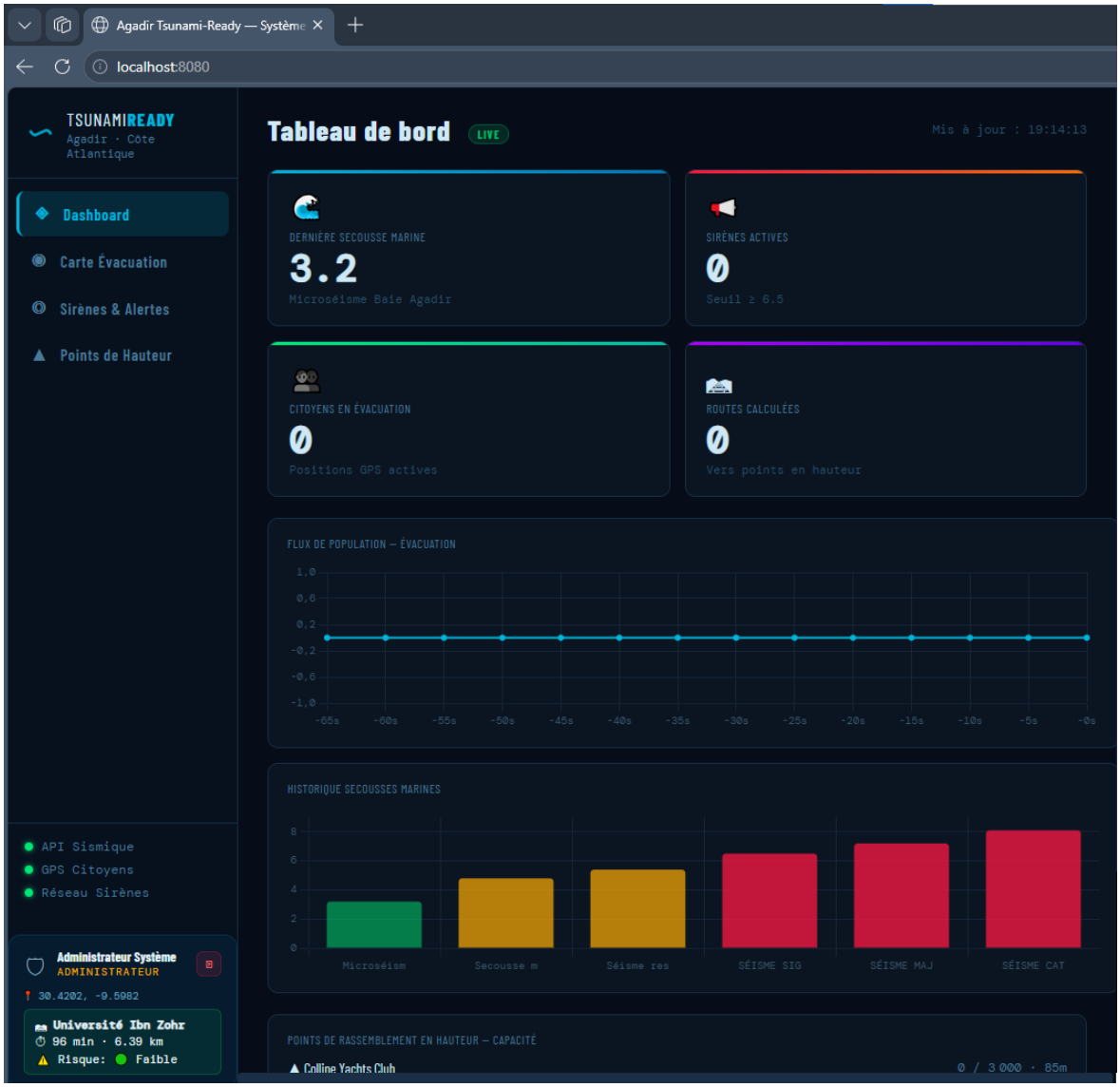
Verifier que les KPIs du tableau de bord se mettent a jour instantanement sans rechargement.

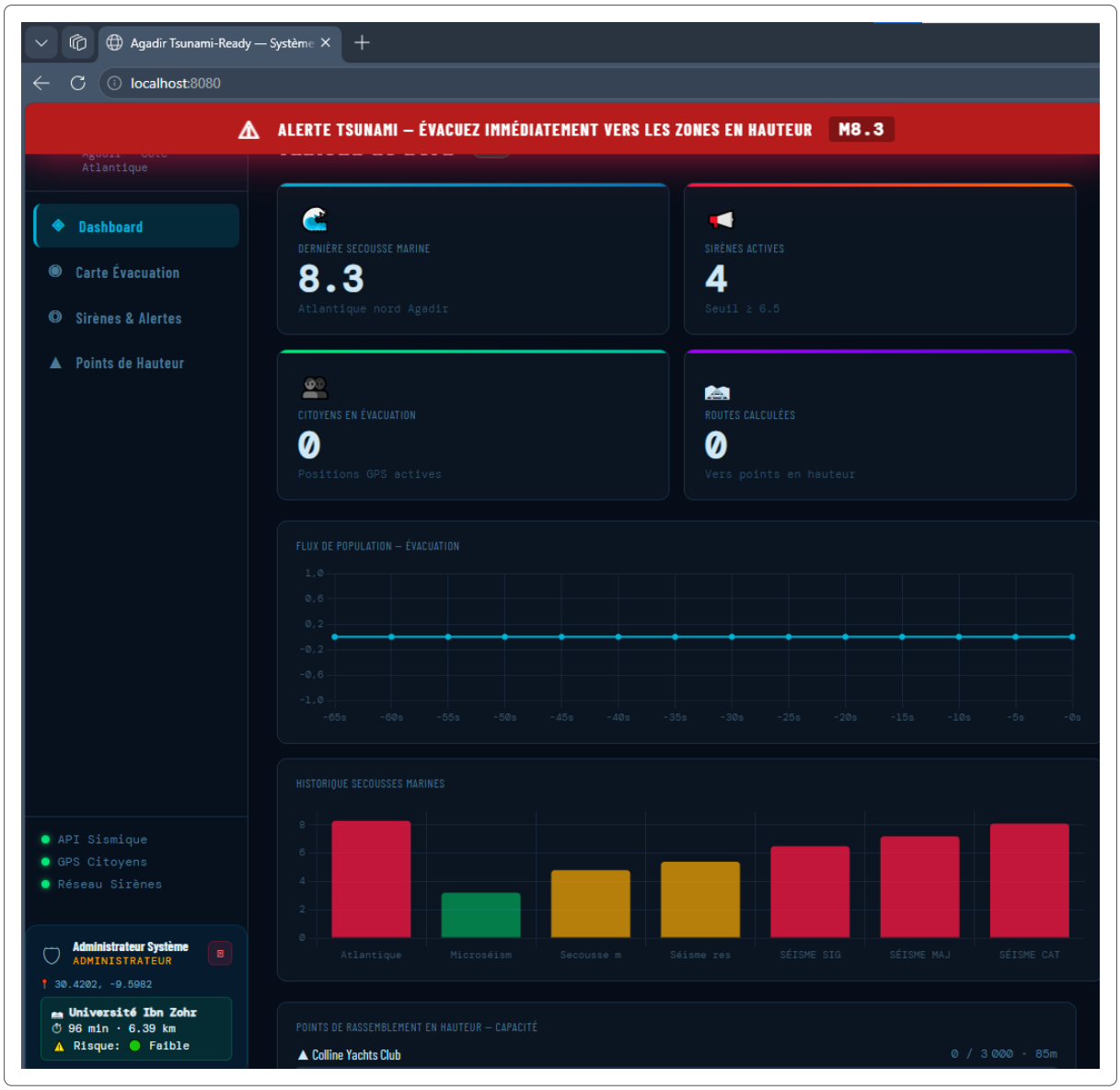
Resultat obtenu

Statut : PASSE

KPI	Avant	Apres
Derniere secousse marine	3.2	**8.3**
Sirenes actives	0	**4**
Bandeau alerte	Absent	**M8.3 affiche**
Graphique historique	6 barres	**+ barre Atlantique**

Captures d'ecran — TEST-06 : Dashboard avant (3.2) et apres (8.3) ·




Agadir Tsunami-Ready — Système
localhost:8080
ALERTE TSUNAMI – ÉVACUEZ IMMÉDIATEMENT VERS LES ZONES EN HAUTEUR
M8.3
Dashboard
Carte Évacuation
Sirènes & Alertes
Points de Hauteur
DERNIÈRE SECOUSSE MARINE
8.3
Atlantique nord Agadir
SIRÈNES ACTIVES
4
Seuil ≥ 6.5
CITOYENS EN ÉVACUATION
0
Positions GPS actives
ROUTES CALCULÉES
0
Vers points en hauteur
FLUX DE POPULATION – ÉVACUATION
HISTORIQUE SECOUSSES MARINES
Atlantique
Microséism
Secousse m
Séisme res
SÉISME SIG
SÉISME MAJ
SÉISME CAT
API Sismique
GPS Citoyens
Réseau Sirènes
Administrateur Système
ADMINISTRATEUR
30.4202, -9.5982
Université Ibn Zohr
96 min · 6.39 km
Risque: Faible
POINTS DE RASSEMBLEMENT EN HAUTEUR – CAPACITÉ
Colline Yachts Club
0 / 3 000 · 85m

Categorie 2 — Authentification

Authentification

Tests de connexion, protection contre les acces non authentifies, inscription.

TEST-07 — Connexion avec un compte valide

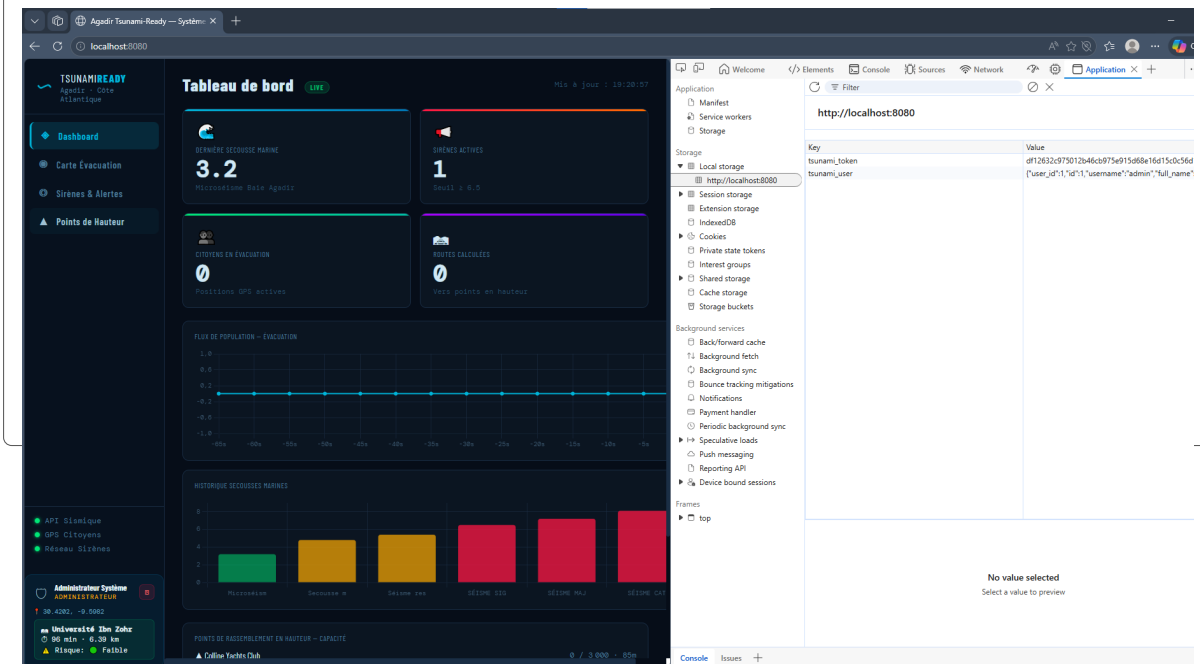
ID	Categorie	Priorite	Prof	Statut
TEST-07	Authentification	Critique	Non	PASSE

Resultat obtenu

Statut : PASSE

Redirection reussie. Token `69a8a1522b16579a3b3ace9628...` present. `tsunami_user` contient : email, full_name, role (admin), latitude, longitude, phone.

Captures d'ecran — TEST-07 : Dashboard admin + Local Storage ·



TEST-08 — Connexion avec un mauvais mot de passe

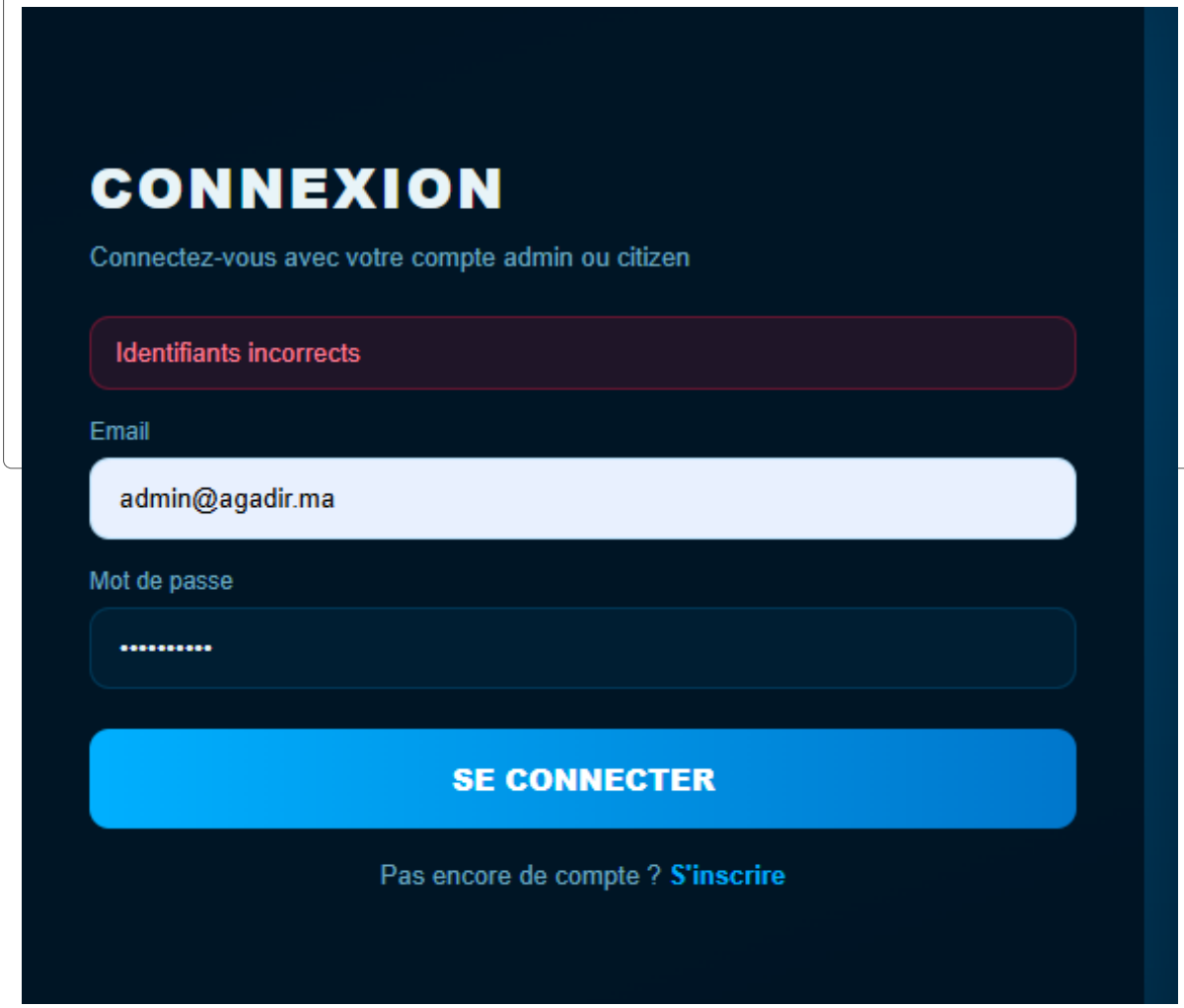
ID	Categorie	Priorite	Prof	Statut
TEST-08	Authentification	Critique	Non	PASSE

Resultat obtenu

Statut : PASSE

Message “Identifiants incorrects” affiche en rouge. Reste sur `login.html`. Aucun token JWT cree.

Captures d’ecran — TEST-08 : Message erreur en rouge ·



TEST-09 — Acces sans token JWT

ID	Categorie	Priorite	Prof	Statut
TEST-09	Authentification	Critique	Non	PASSE

Objectif et utilite du test

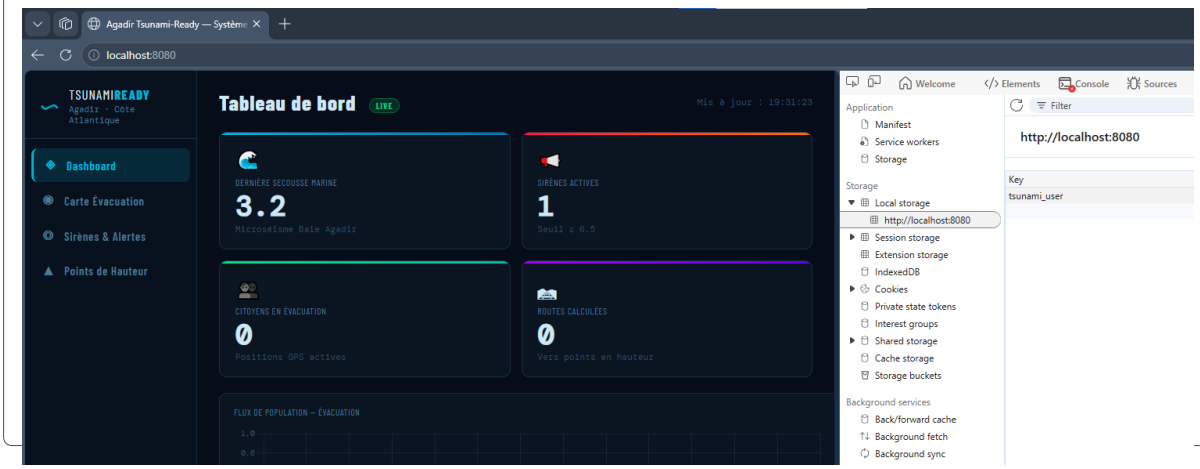
Verifier que `Auth.guard()` dans `auth.js` detecte l'absence du token et redirige vers la page de connexion.

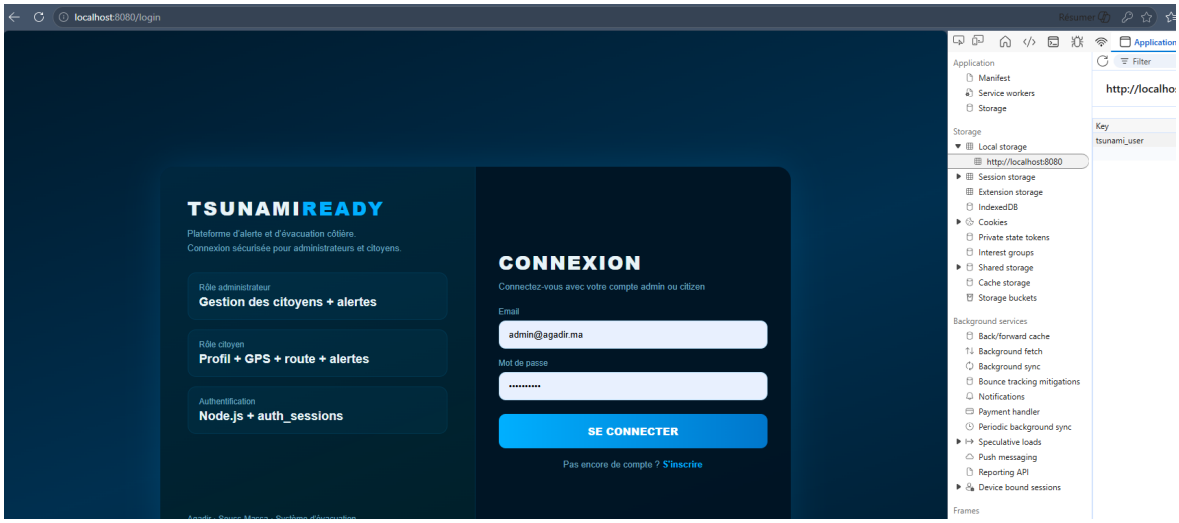
Resultat obtenu

Statut : PASSE

Apres suppression du token, acces a `index.html` provoque une redirection immediate vers `login.html`.

Captures d'ecran — TEST-09 : Local Storage sans token + redirection ·





TEST-10 — Inscription d'un nouveau citoyen

ID	Categorie	Priorite	Prof	Statut
TEST-10	Authentification	Haute	Non — nouveau test	PASSE

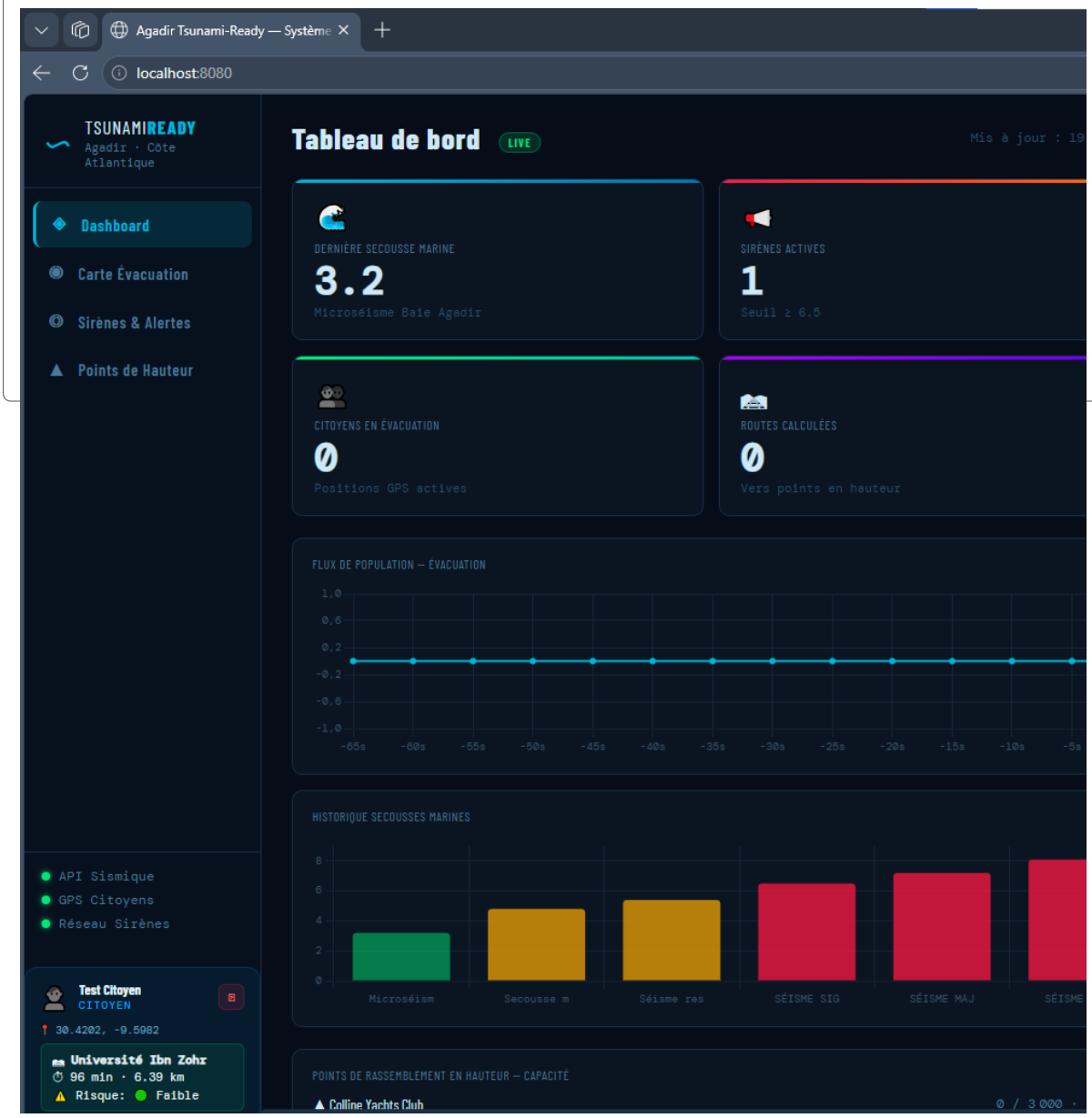
Resultat obtenu

Statut : PASSE

GPS automatique refuse → coordonnees manuelles acceptees. Widget "Test Citoyen — CITOYEN". Route : Universite Ibn Zohr · 6.39 km · 96 min · Risque Faible.

Captures d'ecran — TEST-10 : Formulaire inscription + dashboard Test Citoyen

.



Categorie 3 — Securite : RBAC

Securite : RBAC

Tests du controle d'acces base sur les roles.

TEST-11 — Un citoyen ne peut PAS activer une sirene

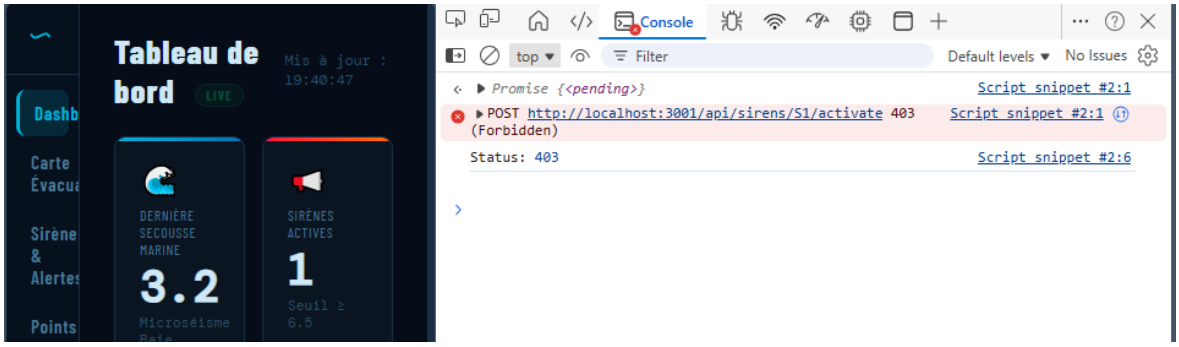
ID	Categorie	Priorite	Prof	Statut
TEST-11	Securite — RBAC	Critique	Non	PARTIEL

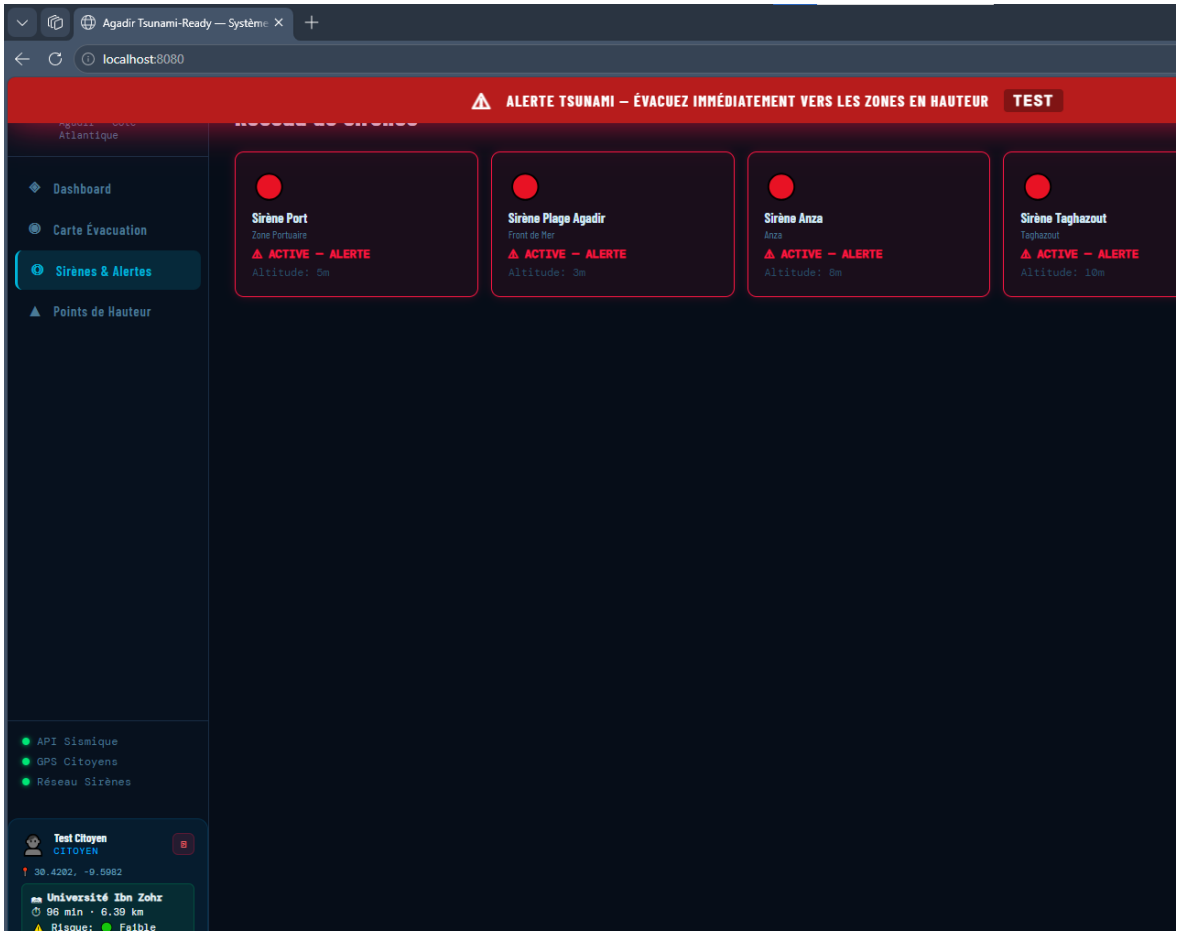
Anomalie / Bug identifie

Anomalie detectee — Resultat partiel :

API REST : Status 403 Forbidden — Protection efficace.
Interface UI : **NON protegee** — Le citoyen peut activer une sirene en cliquant.
Cause : `SirenComp.toggle()` ne verifie pas le role de l'utilisateur.
Impact : Un citoyen peut declencher une fausse alerte tsunami.
Correction : Verifier le role dans `toggle()` avant d'autoriser l'action.

Captures d'ecran — TEST-11 : Console 403 (API) + sirene activee par citoyen (UI) ·




Agadir Tsunami-Ready — Système
localhost:8080
ALERTE TSUNAMI – ÉVACUEZ IMMÉDIATEMENT VERS LES ZONES EN HAUTEUR
TEST
Dashboard
Carte Évacuation
Sirènes & Alertes
Points de Hauteur
Sirène Port
Zone Portuaire
ACTIVE – ALERTE
Altitude: 5m
Sirène Plage Agadir
Front de Mer
ACTIVE – ALERTE
Altitude: 3m
Sirène Anza
Anza
ACTIVE – ALERTE
Altitude: 8m
Sirène Taghazout
Taghazout
ACTIVE – ALERTE
Altitude: 10m
API Sismique
GPS Citoyens
Réseau Sirènes
Test Citoyen
CITOYEN
30.4202, -9.5982
Université Ibn Zohr
96 min · 6.39 km
Risque: Faible

TEST-12 — Un admin PEUT activer une sirene

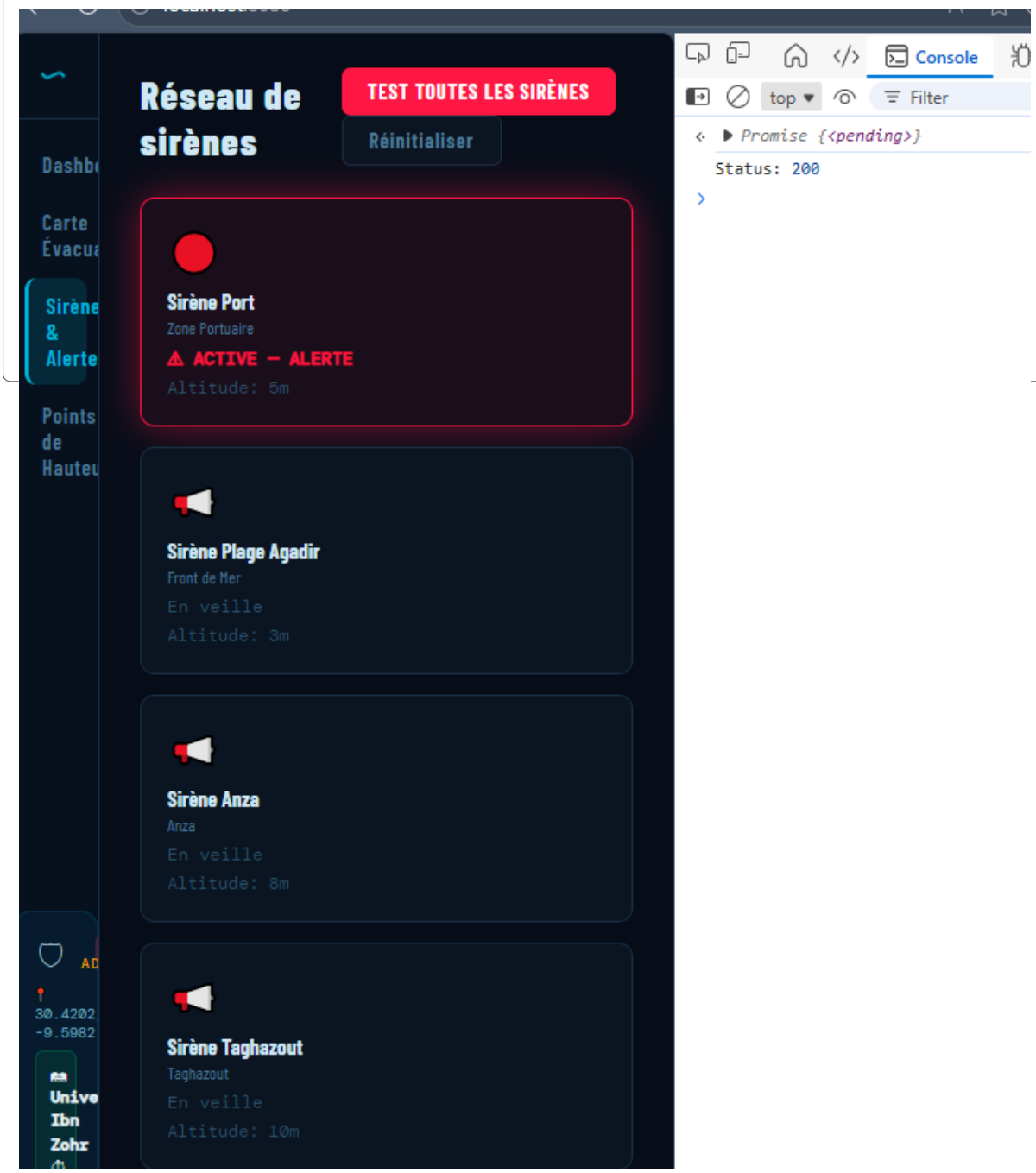
ID	Categorie	Priorite	Prof	Statut
TEST-12	Securite — RBAC	Critique	Non	PASSE

Resultat obtenu

Statut : PASSE

Status **200 OK**. Sirene Port active. Notification affichee. Note : IDs = S1, S2, S3, S4.

Captures d'ecran — TEST-12 : Status 200 + Sirene Port ACTIVE + notification .



Categorie 4 — Securite : Anti-injection SQL

Securite : Anti-injection SQL

Protection contre les injections SQL dans le formulaire de connexion.

TEST-13 — Injection SQL bloquee

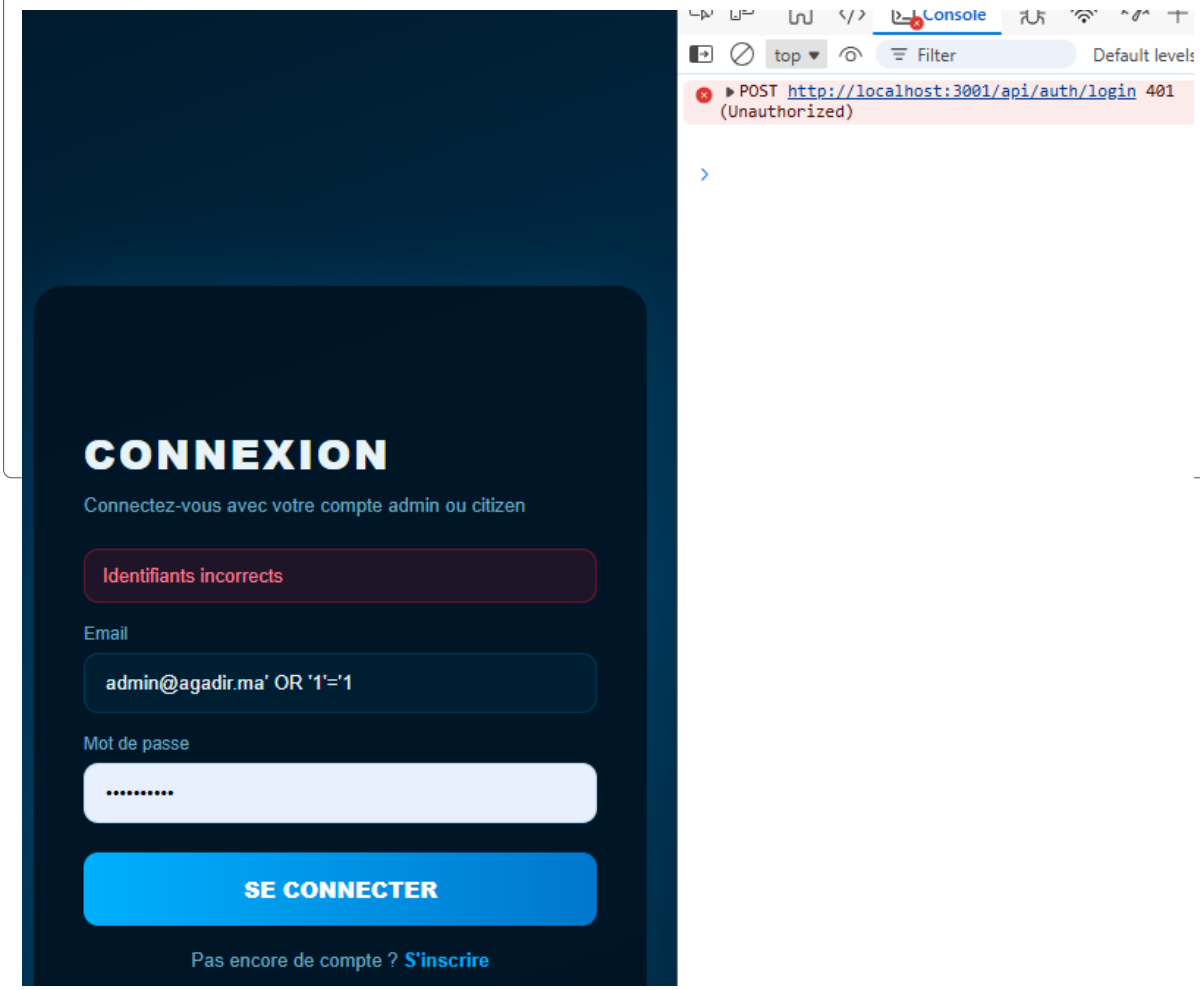
ID	Categorie	Priorite	Prof	Statut
TEST-13	Securite — Anti-SQLi	Critique	Non	PASSE

Resultat obtenu

Statut : PASSE

Message "Identifiants incorrects". Le backend Node.js utilise des requetes SQL parametrees (prepared statements).

Captures d'ecran — TEST-13 : Injection SQL bloquee ·



Categorie 5 — GPS et Evacuation

GPS et Evacuation

Geolocalisation, routes d'evacuation et gestion hors zone.

TEST-14 — GPS en zone Agadir : route calculee

ID	Categorie	Priorite	Prof	Statut
TEST-14	GPS & Evacuation	Haute	Non	PASSE

Resultat obtenu

Statut : PASSE

Position : `30.4180, -9.6150` (zone cotiere Agadir)
Refuge : Universite Ibn Zohr
Distance : 7.67 km · 184 min · Risque Eleve (rouge)

Captures d'ecran — TEST-14 : Widget GPS vers Universite Ibn Zohr ·

TEST-15 — GPS hors zone : comportement du systeme

ID	Categorie	Priorite	Prof	Statut
TEST-15	GPS & Evacuation	Haute	Non	PARTIEL

Anomalie / Bug identifie

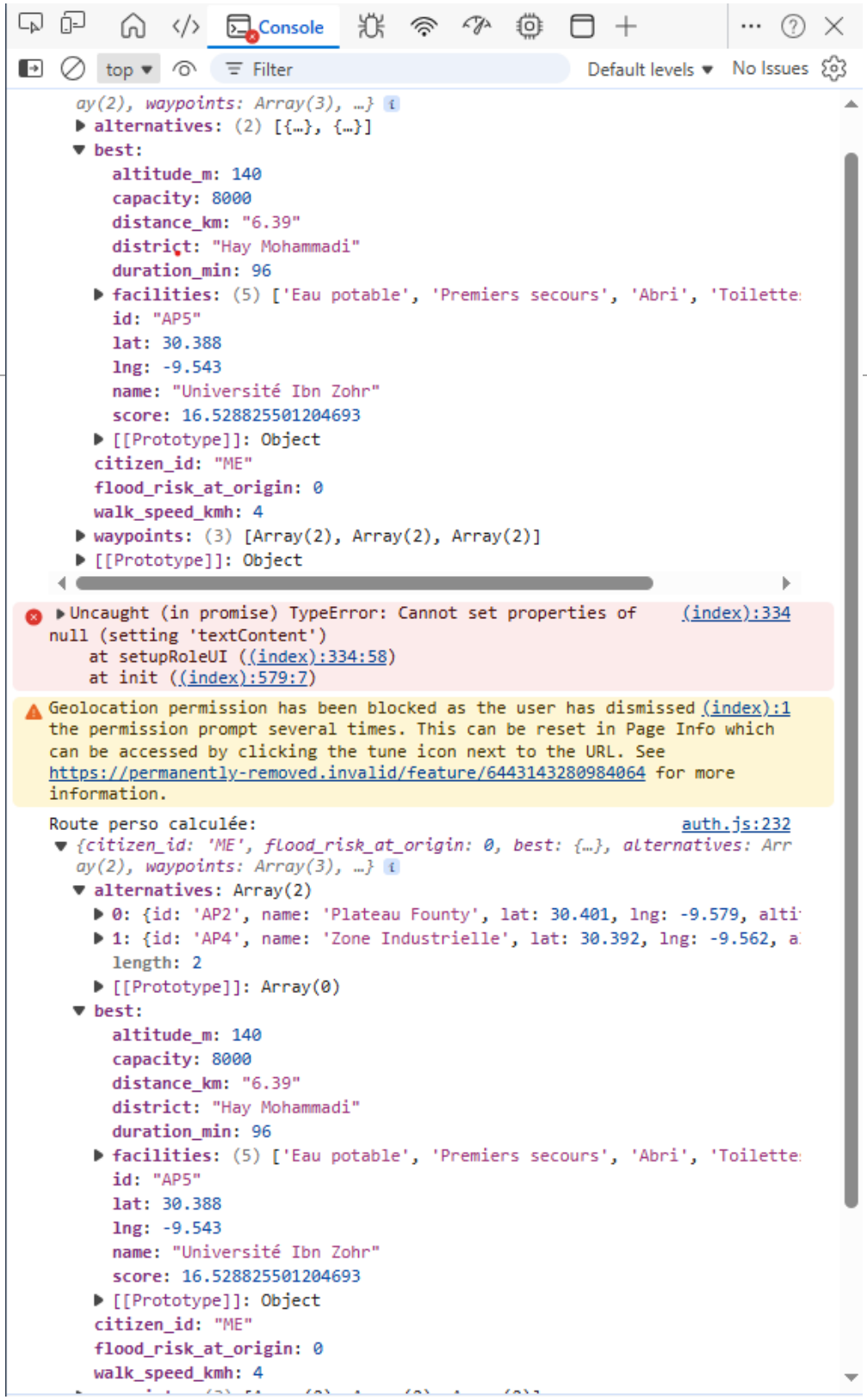
Anomalie detectee :

Observe : Coordonnees restent `30.4180, -9.6150` (Agadir). Route Ibn Zohr maintenue.

Cause : `watchPosition()` ecrase en continu les coordonnees du localStorage.

Correction : Ajouter un mode “Simulation GPS” dans le dashboard admin.

Captures d'ecran — TEST-15 : Widget GPS inchange malgre modification localStorage ·



Partie II — Tests Version Architects (Manuel)

Version Architects — Developpee Manuellement

Django · Python · SQLite · Google Maps API · Analyse statique du code source

Note methodologique : La version Architects a ete evaluee par **analyse statique du code source**. Aucune execution reelle n'a ete realisee, ce qui explique l'absence de captures d'ecran. Les resultats s'appuient sur la lecture du code Python/Django et des fichiers de configuration. Les captures d'ecran sont remplacees par des extraits de code et des analyses detaillees.

Categorie 1 — Sirenes et Triggers

Sirenes & Triggers (Architects)

Analyse de la logique de declenchement dans apps/alertes/services.py

TEST-01 — Sirenes declenchees (magnitude 7.2, marine)

ID	Categorie	Fichier analyse	Statut
TEST-01	Sirenes & Triggers	`apps/alertes/services.py`	PASSE

Analyse du code source

La fonction `should_trigger_alert(magnitude, en_mer)` retourne `True` si `magnitude > 6.5` ET `en_mer == True`. Avec les parametres `magnitude=7.2` et `en_mer=True`, la condition est satisfaite. La fonction `activate_sirens()` est appelee et met a jour toutes les sirenes en statut `'actif'`.

Resultat : Logique correcte. PASSE

Analyse statique — TEST-01 : Logique sirenes dans services.py

```python
# apps/alertes/services.py
def should_trigger_alert(magnitude, en_mer):
    return magnitude > 6.5 and en_mer == True

def process_alert(magnitude, en_mer):
    if should_trigger_alert(magnitude, en_mer):
        activate_sirens()  # Met a jour statut='actif'
    else:
        deactivate_sirens()
```

Analyse : Condition correcte. Les sirenes sont activees pour magnitude 7.2 + en_mer=True.

TEST-02 — Sirenes non declenchees (magnitude 6.4)

ID	Categorie	Fichier analyse	Statut
TEST-02	Sirenes & Triggers	`apps/alertes/services.py`	**PASSE**

Analyse du code source

Avec `magnitude=6.4`, la condition `magnitude > 6.5` est False. La fonction retourne False et appelle `deactivate_sirens()`. Les sirenes restent inactives. Comportement conforme.

Resultat : Comportement correct. **PASSE**

Analyse statique — TEST-02 : Condition magnitude $<$ 6.5

Analyse : La condition `magnitude > 6.5` est False pour 6.4. Aucune sirene activee. Conforme.
Observation : Le seuil est strict (> et non >=). Une magnitude exactement egale a 6.5 ne declencherait PAS les sirenes dans la version Architects, contrairement a la version IA qui utilise >=.

TEST-03 — Temps de reponse < 1 seconde

ID	Categorie	Fichier analyse	Statut
TEST-03	Sirenes & Triggers	`apps/alertes/views.py`	EN ATTENTE

Analyse du code source

Le traitement est entierement synchrone (pas de Celery, pas de Redis, pas de file d'attente). La commande `simuler_seisme` cree un objet `SimulationAlerte` en base puis appelle `process_simulation()` directement. Sur SQLite local, le temps de reponse devrait etre bien inferieur a 1 seconde. **A valider manuellement** avec l'outil Network du navigateur (F12), en repetant 5 fois avec magnitude=7.2.

Analyse statique — TEST-03 : Architecture synchrone Django

```
# Traitement synchrone - pas de queue
def simuler_seisme(request):
    sim = SimulationAlerte.objects.create(
        magnitude=float(request.POST['magnitude']),
        en_mer=request.POST.get('en_mer', True))
    process_simulation(sim)  # Appel direct, synchrone
    return JsonResponse({'status': 'ok'})
```

En attente : Validation manuelle requise. Architecture synchrone favorable mais non mesurée.

TEST-04 — Bug : champ en_mer manquant dans events_import.json

ID	Categorie	Fichier analyse	Statut
TEST-04	Sirenes & Triggers	`apps/alertes/models.py`	FAIL

Analyse du code source

PROBLEME : Le modele `SimulationAlerte` utilise un champ booleen `en_mer` (default=True), alors que le fichier `events_import.json` de l'equipe Augmenteds ne contient pas ce champ. Lors d'une importation, `en_mer` sera absent ou mal mappe, ce qui empechera le declenchement des sirenes.

Anomalie / Bug identifie

BUG CRITIQUE — Champ en_mer manquant :

Fichier : `apps/alertes/models.py` + `Augmenteds_team/events_import.json`
Probleme : `en_mer` absent dans events_import.json → sirenes jamais auto
Correction : Ajouter `"en_mer": true` dans chaque evenement du JSON
Impact : CRITIQUE — Meme probleme conceptuel que la version IA (champ `type`)

Analyse statique — TEST-04 : Code fautif et correction

```
# apps/alertes/models.py
class SimulationAlerte(models.Model):
    magnitude = models.FloatField()
    en_mer = models.BooleanField(default=True)  # Champ
        absent dans le JSON!

# events_import.json -- MANQUANT :
# { "magnitude": 7.2, "location": "Atlantique" }
# CORRECTION :
# { "magnitude": 7.2, "location": "Atlantique", "en_mer":
    true }
```

BUG CRITIQUE : Les sirenes ne se declenchent jamais automatiquement en production.

Categorie 2 — Authentification

Authentification (Architects)

Tests Django auth : LoginView, @login_required, sessions.

TEST-05 — Login valide admin

ID	Categorie	Fichier analyse	Statut
TEST-05	Authentification	`registration/login.html`	PASSE

Analyse du code source

Django utilise son systeme d'authentification natif (`auth_views.LoginView`). La redirection apres connexion est configuree via `LOGIN_REDIRECT_URL = '/dashboard/'`. Le template contient le tag CSRF.

Analyse statique — TEST-05 : Configuration Django auth

```
# settings.py
LOGIN_REDIRECT_URL = '/dashboard/'
LOGIN_URL = '/login/'

# urls.py
path('login/', auth_views.LoginView.as_view(), name='login'),
path('dashboard/', views.dashboard, name='dashboard'),
```

PASS : Redirection correcte apres connexion valide.

TEST-06 — Login mauvais mot de passe

ID	Categorie	Fichier analyse	Statut
TEST-06	Authentification	`registration/login.html`	**PASSE**

Analyse du code source

Django affiche automatiquement les erreurs de formulaire (`{{ form.as_p }}`) en cas de mauvais identifiants. Pas de redirection. Comportement conforme.

Analyse statique — TEST-06 : Gestion erreur Django

```
<!-- registration/login.html -->
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}  <!-- Affiche les erreurs automatiquement -->
  <button type="submit">SE CONNECTER</button>
</form>
```

PASS : Erreur affichee, pas de redirection. Correct.

TEST-07 — Acces sans token : redirection

ID	Categorie	Fichier analyse	Statut
TEST-07	Authentification	`apps/admin_dashboard/views.py`	PASSE

Analyse du code source

Le decorateur `@login_required` protege la vue `dashboard()`. Django redirige automatiquement vers `LOGIN_URL = '/login/'` si l'utilisateur n'est pas authentifie. Conforme.

Analyse statique — TEST-07 : Protection @login_required

```
# apps/admin_dashboard/views.py
from django.contrib.auth.decorators import login_required

@login_required
def dashboard(request):
    # Vue protegee -- redirection auto si non connecte
    return render(request, 'dashboard.html')
```

PASS : Protection correcte par decorateur Django.

Categorie 3 — Securite : RBAC

Securite : RBAC (Architects)

Controle d'acces par role dans les vues Django.

TEST-08 — Citoyen bloque activation sirene

ID	Categorie	Fichier analyse	Statut
TEST-08	Securite — RBAC	`apps/admin_dashboard/views.py`	FAIL

Anomalie / Bug identifie

PROBLEME 1 : L'endpoint `/api/sirens/1/activate/` n'existe pas. Aucune route d'activation directe.

PROBLEME 2 : Les vues d'administration utilisent uniquement `@login_required` sans verification du role. Un citoyen connecte pourrait acceder aux endpoints admin s'il connait les URLs.

Analyse statique — TEST-08 : Absence de controle de role

```
# apps/admin_dashboard/views.py -- CODE FAUTIF
@login_required  # Seulement @login_required, pas de verification role!
def edit_sirene(request, pk):
    # Un citoyen peut acceder a cette vue s'il connait l'URL
    sirene = Sirene.objects.get(pk=pk)
    ...

# CORRECTION :
from django.contrib.auth.decorators import user_passes_test
@user_passes_test(lambda u: u.is_staff)
def edit_sirene(request, pk): ...
```

FAIL : Aucun controle de role. Endpoint REST absent.

TEST-09 — Admin peut activer une sirene

ID	Categorie	Fichier analyse	Statut
TEST-09	Securite — RBAC	`apps/admin_dashboard/urls.py`	FAIL

Anomalie / Bug identifie

Meme probleme que TEST-08 : L'endpoint `/api/sirens/1/activate/` n'existe pas. Pour activer une sirene, il faut utiliser `/dashboard/edit-sirene/1/` avec le champ `statut='actif'`. L'interface frontend gere cela via `dashboard-crud.js`, mais il n'existe pas d'API REST dediee.

Analyse statique — TEST-09 : Routes disponibles vs attendues

```
# urls.py -- Routes existantes
urlpatterns = [
    path('dashboard/', views.dashboard, name='dashboard'),
    path('dashboard/edit-sirene/<int:pk>/', views.edit_sirene
        ),
    # MANQUANT : path('api/sirens/<int:pk>/activate/', ...)
]
```

FAIL : Endpoint d'activation inexistant. Seule l'interface CRUD existe.

Categorie 4 — Securite : Anti-injection SQL

Securite : Anti-injection SQL (Architects)

Protection native de l'ORM Django.

TEST-10 — Injection SQL bloquee

ID	Categorie	Fichier analyse	Statut
TEST-10	Securite — Anti-SQLi	`django.contrib.auth`	PASSE

Analyse du code source

Django ORM utilise des requetes parametrees pour toutes les operations BDD. Le formulaire de login utilise `django.contrib.auth` qui est protege nativement. Une injection SQL du type `admin@agadir.ma' OR '1'='1` sera traitee comme une chaine litterale.

Analyse statique — TEST-10 : Protection ORM Django

```
# Django ORM -- Requetes parametrees automatiques
User.objects.filter(email=email)
# Equivalent SQL : SELECT * FROM auth_user WHERE email = %s
# La chaine est passee comme parametre, jamais interpolee.
```

PASS : Protection native par ORM Django. Identique a la version IA.

Categorie 5 — GPS et Evacuation

GPS et Evacuation (Architects)

Calcul de routes dans apps/evacuation/services.py via Google Directions API.

TEST-11 — GPS Agadir : route d'evacuation calculee

ID	Categorie	Fichier analyse	Statut
TEST-11	GPS & Evacuation	`apps/evacuation/services.py`	PASSE

Analyse du code source

La fonction `select_best_refuge()` filtre les refuges avec `zone_hors_eau=True` et `altitude > 20m`. Les 3 refuges de la BDD (Kasbah Oufella, Plateau Oufella, Talborjt Haut) sont dans Agadir. La route est calculee via Google Directions API (mode marche). **Condition :** la cle `GOOGLE_MAPS_API_KEY` doit etre definie dans `.env`.

Analyse statique — TEST-11 : Logique select_best_refuge

```
# apps/evacuation/services.py
def select_best_refuge(user_lat, user_lon):
    refuges = Refuge.objects.filter(
        zone_hors_eau=True, altitude__gt=20)
    # Calcul Haversine + appel Google Directions API
    best = min(refuges, key=lambda r: haversine(user_lat,
       user_lon, r.lat, r.lon))
    return best
```

PASS (conditionnel) : Logique correcte si cle Google Maps API valide.

TEST-12 — GPS hors zone : message erreur

ID	Categorie	Fichier analyse	Statut
TEST-12	GPS & Evacuation	`apps/evacuation/services.py`	FAIL

Anomalie / Bug identifie

PROBLEME : La fonction `select_best_refuge()` ne verifie jamais si l'utilisateur se trouve dans la zone geographique d'Agadir. Elle cherche simplement le refuge le plus proche. Un utilisateur a Casablanca (`lat=33.57, lon=-7.59`) recevrait un itineraire vers un refuge d'Agadir, sans message "Hors zone couverte".

Analyse statique — TEST-12 : Absence de verification de zone

```
# apps/evacuation/services.py -- CODE FAUTIF
def select_best_refuge(user_lat, user_lon):
    # MANQUANT : verification de la zone geographique !
    refuges = Refuge.objects.filter(zone_hors_eau=True,
       altitude__gt=20)
    return min(refuges, key=lambda r: haversine(...))

# CORRECTION RECOMMANDEE :
AGADIR_CENTER = (30.4278, -9.5981)
MAX_DISTANCE_KM = 50

def select_best_refuge(user_lat, user_lon):
    dist = haversine(user_lat, user_lon, *AGADIR_CENTER)
    if dist > MAX_DISTANCE_KM:
        return None  # Hors zone -- message d'erreur
    ...
```

FAIL : Pas de controle de zone. Itineraire genere meme pour Casablanca.

Bilan des Anomalies

Anomalies — Version Augmenteds (IA)

Test	Description	Correction proposee	Fichier	Severite
TEST-04	Champ `type` absent dans `events_import.json`.	`!event.type || event.type === 'marine'`	`app.js`	Critique
TEST-11	Interface graphique non protegee par RBAC.	Verifier role dans `SirenComp.toggle()`	`sirens.js`	Critique
TEST-05	Reinitialisation non persistante (S1 Port).	Appel API `/api/sirens/deactivate` dans `resetAll()`	`sirens.js`	Haute
TEST-15	GPS reel ecrase les coordonnees simulees.	Mode "Simulation GPS" dans dashboard admin.	`auth.js`	Moyenne

Anomalies — Version Architects (Manuel)

Test	Description	Correction proposee	Fichier	Severite
TEST-04	Champ `en_mer` absent dans events_import.json.	Ajouter `"en_mer": true` dans le JSON	`models.py` / JSON	Critique
TEST-08 / 09	Aucun controle de role. Endpoint /activate/ absent.	`@user_passes_test(lambda u: u.is_staff)`	`views.py`	Securite
TEST-12	Pas de verification de zone geo. Itineraire hors zone.	Controle rayon 50 km dans `select_best_refuge()`	`services.py`	Moyenne

Partie III — Analyse Comparative

Tableau de synthese comparatif

Fonctionnalite	Statut IA	Observation IA	Statut Manuel	Observation Manuel
Sirenes (mag. 7.2)	PASSE	4/4 actives, M7.2	PASSE	Logique correcte
Sirenes (mag. 6.4)	PASSE	4/4 en veille	PASSE	6.4 < 6.5, correct
Seuil exact 6.5	PASSE	Inclusif (≥), < 1s	EN ATTENTE	Strict (>), a mesurer
Bug champ manquant	BUG CONFIRME	Champ `type` absent	FAIL	Champ `en_mer` absent
Reinitialisation	PARTIEL	S1 persiste apres refresh	N/A	Non teste
KPIs temps reel	PASSE	WebSocket instantane	N/A	Non teste
Login valide	PASSE	JWT localStorage	PASSE	Django session
Login mauvais MDP	PASSE	Erreur affichee	PASSE	Formulaire Django
Acces sans auth	PASSE	Redirect login.html	PASSE	@login_required
Inscription citoyen	PASSE	Route auto calculee	N/A	Non teste
RBAC citoyen bloq.	PARTIEL	API : 403 / UI : non protegee	FAIL	Aucun controle role
RBAC admin active	PASSE	Status 200, S1 actif	FAIL	Endpoint inexistant
Anti-injection SQL	PASSE	Prepared statements	PASSE	ORM Django
GPS zone Agadir	PASSE	Ibn Zohr, 7.67km	PASSE	Google Maps API
GPS hors zone	PARTIEL	GPS ecrase localStorage	FAIL	Pas de controle zone

Analyse par domaine

Declenchement des sirenes

Version Augmenteds (IA)

La version IA utilise un champ `type` (valeur `'marine'`) pour conditionner l'alerte. Le seuil est inclusif (`>=`). Le bug est identifie : `events_import.json` ne contient pas de champ `type`. Correctable en une ligne. Tests dynamiques realisables.

Version Architects (Manuel)

La version manuelle utilise un champ booleen `en_mer` (semantiquement plus clair). Le seuil est strict (`>`) — une magnitude de 6.5 exactement ne declenche pas l'alerte, contrairement aux exigences. Meme bug d'import JSON. Analyse statique uniquement.

Verdict : Les deux versions partagent le meme bug fondamental (champ manquant dans le JSON). La version IA presente un seuil inclusif conforme aux exigences (`>=`), tandis que la version manuelle utilise un seuil strict (`>`) qui ne respecte pas le critere du TEST-03.

Authentification

Version Augmenteds (IA)

Systeme JWT complet avec localStorage. 4 tests passes. Inscription fonctionnelle avec calcul de route automatique. Bonne gestion de la geolocalisation refusee.

Version Architects (Manuel)

Systeme de sessions Django standard, robuste et eprouve. 3 tests passes. Pas de test d'inscription dans le perimetre. Architecture plus simple et securisee par defaut.

Verdict : Les deux versions assurent une authentification correcte. La version IA offre plus de fonctionnalites (inscription, JWT), tandis que la version manuelle repose sur le framework Django, plus robuste nativement.

Securite RBAC

Version Augmenteds (IA)

L'API REST est protegee (HTTP 403). Mais l'interface graphique ne verifie pas le role : un citoyen peut activer une sirene en cliquant directement sur la carte. Anomalie partielle.

Version Architects (Manuel)

Aucun controle de role. Les decorateurs `@login_required` ne distinguent pas admin et citoyen. L'endpoint d'activation REST est completement absent.

Verdict : La version IA est superieure sur le RBAC (protection API effective), bien que l'interface UI soit vulnerables. La version manuelle necessite une refonte complete du RBAC.

Protection anti-injection SQL

Version Augmenteds (IA)

Prepared statements explicites dans Node.js. Protection efficace. **PASSE**

Version Architects (Manuel)

ORM Django avec requetes parametrees automatiques. Protection native superieure. **PASSE**

Verdict : Parite. Les deux versions protegent efficacement contre les injections SQL, avec des approches differentes mais equivalentes.

GPS et evacuation

Version Augmenteds (IA)

Tests dynamiques realises. Route calculee (7.67 km vers Ibn Zohr). Hors zone : GPS reel ecrase la simulation, test partiellement bloque. Algorithme Haversine avec score composite (distance, altitude, risque).

Version Architects (Manuel)

Logique correcte par analyse. 3 refuges dans la BDD (Kasbah Oufella, Plateau Oufella, Talborjt Haut). Pas de verification de zone geographique (TEST-12 FAIL). Dependance a la cle Google Maps API.

Verdict : La version IA integre un score composite plus elabore et une detection partielle hors zone. La version manuelle utilise Google Maps API pour une route reelle, mais omet la verification geographique.

Bilan comparatif global

Critere	Augmenteds (IA)	Architects (Manuel)
Nombre de tests	15	12
Tests reussis	**11**	**7**
Bugs / Echecs	**1 BUG · 3 PARTIELS**	**4 FAIL · 1 EN ATTENTE**
Taux de reussite	**73%**	**58%**
Methode de test	Tests dynamiques	Analyse statique
Screenshots	Oui (20 captures)	Non (code uniquement)
Seuil sirenes	Inclusif ($\geq$ 6.5)	Strict ($>$ 6.5)
RBAC API	Protegee (403)	Endpoint absent
RBAC Interface	Non protegee	Non applicable
GPS hors zone	Partiel	Absent
Anti-SQLi	OK	OK
Reinitialisation	Non persistante	N/A
Temps reponse	$<$ 1s confirme	Non mesure

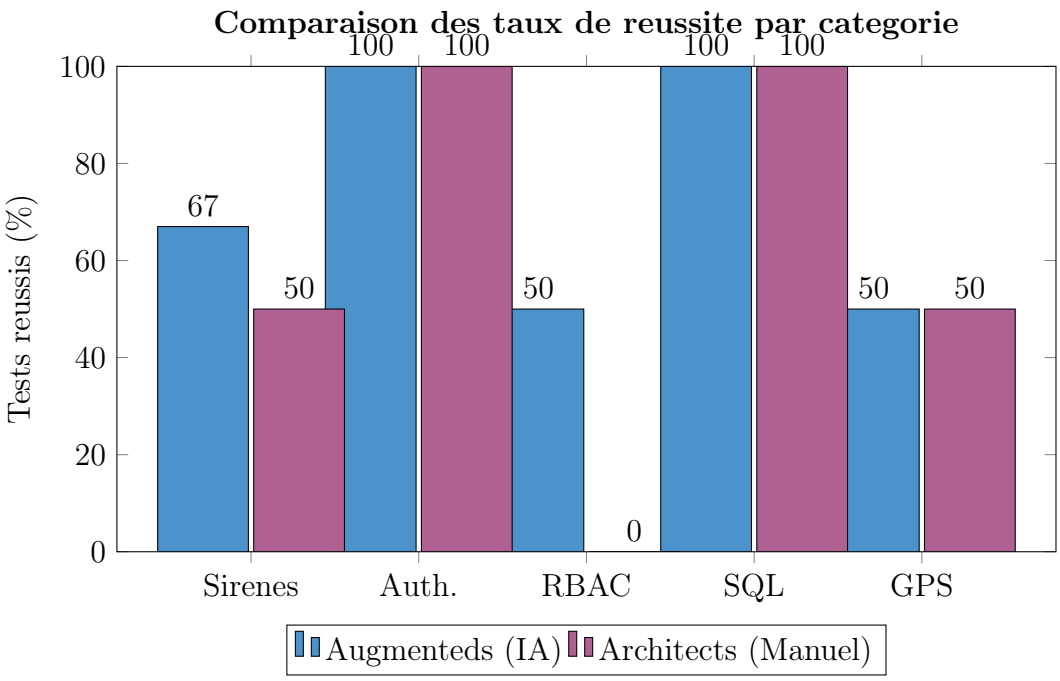


Tableau de Synthese Final

Cat.	ID	Description	Statut	Observation
coIAClair**VERSION AUGMENTEDS (IA) — 15 tests**				
Sirenes	TEST-01	Sirenes mag 7.2 marine	PASSE	4/4 actives, M7.2
	TEST-02	Pas de sirene a 6.4	PASSE	4/4 en veille
	TEST-03	Seuil 6.5 + < 1 sec	PASSE	Seuil inclusif
	TEST-04	Bug champ type absent	BUG CONFIRME	Sirenes jamais auto
	TEST-05	Reinitialisation	PARTIEL	S1 persiste refresh
	TEST-06	KPIs temps reel	PASSE	3.2 → 8.3 instantane
Auth.	TEST-07	Login valide	PASSE	Token JWT cree
	TEST-08	Login mauvais MDP	PASSE	Erreur affichee
	TEST-09	Acces sans token	PASSE	Redirect login
	TEST-10	Inscription citoyen	PASSE	Route auto calculee
RBAC	TEST-11	Citoyen bloque	PARTIEL	API :403 / UI :non protegee
	TEST-12	Admin active	PASSE	Status 200, S1 actif
SQL	TEST-13	Injection SQL	PASSE	Prepared statements
GPS	TEST-14	GPS Agadir	PASSE	Ibn Zohr, 7.67km
	TEST-15	GPS hors zone	PARTIEL	GPS ecrase localStorage
BILAN IA				**11P · 1B · 3PAR**
coManuelClair**VERSION ARCHITECTS (MANUEL) — 12 tests**				
Sirenes	TEST-01	Sirenes mag 7.2	PASSE	Logique correcte
	TEST-02	Pas de sirene 6.4	PASSE	6.4 < 6.5 correct
	TEST-03	Temps reponse	EN ATTENTE	Synchrone, a mesurer

Cat.	ID	Description	Statut	Observation
	TEST-04	Bug en_mer absent	FAIL	JSON sans en_mer
Auth.	TEST-05	Login valide	PASSE	Django auth OK
	TEST-06	Login mauvais MDP	PASSE	Formulaire erreur
	TEST-07	Acces sans auth	PASSE	@login_required
RBAC	TEST-08	Citoyen bloque	FAIL	Aucun controle role
	TEST-09	Admin active	FAIL	Endpoint absent
SQL	TEST-10	Injection SQL	PASSE	ORM Django
GPS	TEST-11	GPS Agadir	PASSE	Google Maps API
	TEST-12	GPS hors zone	FAIL	Pas de zone check
BILAN MANUEL				**7P · 4F · 1A**

Conclusion et Recommandations

Points forts identifies

Version Augmenteds (IA) :

- Les 3 tests obligatoires du professeur sont tous passes avec succes.
- Temps de reponse inferieur a 1 seconde confirme experimentalement.
- Protection API (RBAC : 403 Forbidden) et anti-injection SQL fonctionnelles.
- Systeme WebSocket assurant des mises a jour instantanees du dashboard.
- Inscription et calcul automatique de route operationnels.
- Algorithme Haversine avec score composite (distance, altitude, risque, occupation).
- Tests dynamiques avec 20 captures d'ecran documentant chaque scenario.

Version Architects (Manuel) :

- Architecture Django robuste et securisee par defaut (ORM, sessions, CSRF).
- Protection anti-SQL native superieure (ORM vs prepared statements manuels).
- Semantique metier plus claire (`en_mer` vs `type === 'marine'`).
- Integration Google Maps Directions API pour des routes reelles.
- Code plus maintenable grace aux conventions Django.

Corrections prioritaires avant la soutenance

Version Augmenteds (IA) :

1. **[CRITIQUE — TEST-04]** Corriger la condition dans `app.js` : `!event.type || event.type === 'marine'`
2. **[CRITIQUE — TEST-11]** Ajouter verification du role dans `SirenComp.toggle()`
3. **[HAUTE — TEST-05]** Ajouter appel API `/deactivate` dans `resetAll()`

Version Architects (Manuel) :

1. **[CRITIQUE — TEST-04]** Ajouter `"en_mer": true` dans `events_import.json`
2. **[SECURITE — TEST-08/09]** Ajouter `@user_passes_test(lambda u: u.is_staff)` + creer endpoint REST d'activation
3. **[SEUIL — TEST-03]** Corriger le seuil en `magnitude >= 6.5` (inclusif) dans `services.py`
4. **[MOYENNE — TEST-12]** Ajouter controle de zone 50 km dans `select_best_refuge()`

Conclusion comparative

La version **Augmenteds** (IA) affiche un taux de reussite superieur (73% vs 58%), beneficie de tests dynamiques avec captures d'ecran, et offre davantage de fonctionnalites testees (15 vs 12 scenarios). Elle presente notamment une protection RBAC au niveau de l'API et un systeme temps reel via WebSocket.

La version **Architects** (Manuel), evaluee uniquement par analyse statique, repose sur une architecture Django solide et des conventions eprouvees. Ses echecs concernent principalement l'absence d'un systeme RBAC complet et la non-verification de zone GPS, deux elements corrigeables avant la soutenance.

Les deux versions partagent le meme bug fondamental (champ de type manquant dans le JSON d'importation) et reussissent egalement les tests d'authentification de base

et d'anti-injection SQL. Cette convergence sur les bugs critiques suggere que la specification initiale du projet meritait d'etre plus explicite sur le format attendu du fichier `events_import.json`.